

# Numerical Addendum: McConnell Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

The McConnell calculator is a cutoff sum of closed-form bond kernels. The numerical work is the bond geometry cached for those kernels, the midpoint radius query and filters that decide which candidates enter the sum, the asymmetric per-bond tensor, and the two projection paths used for output. The listings below are distilled from the source: loops and bookkeeping are trimmed, but the constants, guards, tensor entries, and output shapes are the implementation's.

## 1 Bond geometry and candidates

The geometry result builds three arrays indexed by bond: midpoint, length, and axis. For a bond with endpoint positions  $q_A$  and  $q_B$ , the midpoint is their average, the length is  $\ell = |q_B - q_A|$ , and the axis is the normalized endpoint difference. If  $\ell \leq 10^{-15}$ , `Bond::Direction` stores the zero vector. The McConnell kernel consumes that cached vector as it stands; it does not normalize the bond axis again.

```
// Distilled from Bond.h and GeometryResult.cpp.
Vec3 edge = positions[bond.atom_index_b] - positions[bond.atom_index_a];
double len = edge.norm();

conf.bond_lengths[bi] = len;
conf.bond_directions[bi] = (len > 1e-15) ? Vec3(edge / len) : Vec3::Zero();
conf.bond_midpoints[bi] =
    0.5 * (positions[bond.atom_index_a] + positions[bond.atom_index_b]);
```

For each atom, candidate bonds come from the spatial index over bond midpoints. The query radius is the runtime TOML key `mcconnell_bond_anisotropy_cutoff`, 10.0 Å in `data/calculator_params.toml`. The header constant `MCCONNELL_CUTOFF_A` is vestigial; the compute path reads the configuration key shown here. The radius query only chooses candidates. A returned bond still passes the kernel guard and all three filters before it changes any accumulator. The cutoff is not a switching function: a bond outside the query is absent from the per-atom sum, and a returned bond keeps the full  $r^{-3}$  kernel if the later filters accept it. The per-atom path does not apply a second cutoff check after this query.

```
// Distilled from SpatialIndexResult.cpp and McConnellResult.cpp.
auto nearby_bonds =
    spatial.BondsWithinRadius(atom_pos,
        cfg("mcconnell_bond_anisotropy_cutoff")); // 10.0 Å

filters.Add(std::make_unique<MinDistanceFilter>());
filters.Add(std::make_unique<SelfSourceFilter>());
filters.Add(std::make_unique<DipolarNearFieldFilter>());
```

## 2 One bond kernel

For an atom at  $r$ , `disp` is  $x = r - m$ , the displacement from the bond midpoint to the atom. Its norm is  $r$ , and `d_hat` is  $\hat{d} = x/r$ . The cached bond direction is  $b$ : a unit endpoint axis when  $\ell > 10^{-15}$ , and the zero vector when  $\ell \leq 10^{-15}$ . The code forms one dot product,

$$c = \hat{d} \cdot b.$$

For nondegenerate bonds this is  $\cos \theta$ . The helper returns the zero-initialized result before the division by  $r$  when  $r < 0.1$  Å. There is no clamp to 0.1 Å: below the guard the result's tensor, scalar, distance,

and direction stay at zero. At exactly 0.1 Å the formula is evaluated with that distance, so the  $r^{-3}$  scale is  $1000 \text{ Å}^{-3}$ .

```
// Distilled from ComputeBondKernel.
Vec3 disp = atom_pos - bond_midpoint;      // midpoint -> atom
double r = disp.norm();
if (r < cfg("singularity_guard_distance")) return result; // zero

result.distance = r;
double r3 = r * r * r;
Vec3 d_hat = disp / r;
double c = d_hat.dot(bond_direction);

result.f = (3.0 * c * c - 1.0) / r3;

for (int a = 0; a < 3; ++a)
  for (int b = 0; b < 3; ++b)
    result.K(a,b) =
      (3.0*d_hat(a)*d_hat(b) - (a == b ? 1.0 : 0.0)) / r3;

for (int a = 0; a < 3; ++a)
  for (int b = 0; b < 3; ++b)
    result.M_over_r3(a,b) =
      ( 9.0*c*d_hat(a)*bond_direction(b)
        -3.0*bond_direction(a)*bond_direction(b)
        -(3.0*d_hat(a)*d_hat(b) - (a == b ? 1.0 : 0.0)) ) / r3;
```

The stored full tensor is

$$\mathcal{M}_{ab} = \frac{9c\hat{d}_a\hat{d}_b - 3b_ab_b - (3\hat{d}_a\hat{d}_b - \delta_{ab})}{r^3}.$$

The separately stored dipolar kernel is

$$K_{ab} = \frac{3\hat{d}_a\hat{d}_b - \delta_{ab}}{r^3}.$$

Thus the third term of  $\mathcal{M}$  is exactly  $-K$ . For a nondegenerate bond, the second term is also a symmetric dipolar-shaped dyadic, but it includes its trace:

$$\frac{-3\hat{\mathbf{b}} \otimes \hat{\mathbf{b}}}{r^3} = -\frac{3\hat{\mathbf{b}} \otimes \hat{\mathbf{b}} - \mathbf{I}}{r^3} - \frac{\mathbf{I}}{r^3}.$$

After the scalar trace piece is separated, its  $T_2$  part is the negative bond-axis dipolar kernel. The stored  $K$  is the midpoint-to-atom version of the same symmetric-traceless shape.

The three terms map to the tensor split in different ways. For a nondegenerate bond, term 1,  $9c\hat{\mathbf{d}} \otimes \hat{\mathbf{b}}/r^3$ , is generally asymmetric. Its trace is  $9c^2/r^3$ , so it contributes  $3c^2/r^3$  to  $T_0$ ; its antisymmetric part is the only source of  $T_1$ ; and its symmetric traceless remainder contributes to  $T_2$ . Term 2 has trace  $-3/r^3$ , so it contributes  $-1/r^3$  to  $T_0$  and a symmetric-traceless  $T_2$  part, but no  $T_1$ . Term 3 is symmetric traceless, so it contributes only to  $T_2$ . After the distance guard has passed in the degenerate cached-zero-axis case, term 1 and term 2 vanish, and the code evaluates the scalar formula below with  $c = 0$ .

For nondegenerate bonds, those trace pieces explain the scalar. The code computes

$$f = \frac{3c^2 - 1}{r^3}$$

directly, and algebraically

$$f = \hat{b}_a K_{ab} \hat{b}_b = \frac{1}{3} \text{tr } \mathcal{M}.$$

For those bonds,  $f$  is both the double contraction of  $K$  with the bond axis and the single-bond  $T_0$  value of the full McConnell tensor. It is not the trace of  $K$ , which is zero. When the degenerate cached-zero-axis case is evaluated,  $f = -1/r^3$  while  $\text{tr } \mathcal{M} = 0$ .

For nondegenerate bonds, the same  $c$  threads through three places: it is squared in  $f$ , it is the linear weight on the asymmetric first term, and it makes the kernel invariant to reversing the bond endpoints. If  $\hat{b}$  is replaced by  $-\hat{b}$ , then  $c$  also changes sign. The product  $c\hat{b}$ , the dyadic  $\hat{b} \otimes \hat{b}$ , the stored  $K$ , and  $f$  are unchanged.

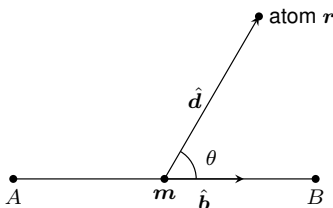


Figure 1: The bond midpoint supplies the expansion centre. The kernel uses the midpoint-to-atom direction  $\hat{d}$  and, for nondegenerate bonds, the cached bond axis  $\hat{b}$ ; their dot product is  $c = \cos \theta$ .

### 3 Guards and filters

After the helper returns, the per-atom path builds a `KernelEvaluationContext`. Its distance field is the helper’s midpoint-to-atom distance, `source_extent` is the cached bond length  $\ell$ , and the two source atom fields are the bond endpoints. The filter set is ordered as minimum distance, self source, then near field. If any filter rejects, the candidate leaves the computation at that point: no scalar sum, tensor sum, bond-neighbour record, nearest-bond record, or category total is updated.

```
KernelEvaluationContext ctx;
ctx.distance      = kernel.distance;
ctx.source_extent = conf.bond_lengths[bi];
ctx.atom_index   = ai;
ctx.source_atom_a = bond.atom_index_a;
ctx.source_atom_b = bond.atom_index_b;

if (!filters.AcceptAll(ctx)) continue;
```

`MinDistanceFilter` requires

$$r \geq \text{singularity\_guard\_distance} = 0.1 \text{ \AA}.$$

For  $r < 0.1 \text{ \AA}$ , the helper has already returned a zero result with `distance == 0`; this filter keeps that zero-valued placeholder from being stored as a real neighbour. At  $r = 0.1 \text{ \AA}$ , the helper has computed the kernel and this filter accepts it, subject to the later filters.

`SelfSourceFilter` is topological. If the target atom is either endpoint of the bond, the bond is omitted even when the midpoint distance passes the numeric tests. The filter changes only membership in the sum; it does not alter the kernel value for other atoms.

`DipolarNearFieldFilter` requires

$$r > \text{near\_field\_exclusion\_ratio} \ell = 0.5 \ell.$$

At or inside half the bond length from the midpoint, the point-source form is not added to the per-atom sums. Equality is rejected on this path. As with the other filters, the skipped contribution is not tapered, clamped, or reweighted into another term.

`SampleKernelAt` evaluates the same bond formula at a free point in space. That path loops over all bonds and has no self-source test because the point has no atom index. It keeps the same singularity guard,

skips  $r > 10.0 \text{ \AA}$ , and skips  $r < 0.5\ell$ . Therefore equality is kept at the sampling cutoff and at the near-field boundary on the free-point path, while the per-atom filter path rejects equality at the near-field boundary.

The nearest peptide C=O direction stored on an atom is not the C=O bond axis. It is  $\hat{d}$ , the midpoint-to-atom direction from the nearest accepted C=O kernel. The code renormalizes that vector before storage; if its norm is not greater than `near_zero_vector_norm_threshold`,  $10^{-10}$ , the stored direction is the zero vector.

```
if (best_co_dist < NO_DATA_SENTINEL) {
  double dir_norm = best_co_direction.norm();
  ca.dir_nearest_CO =
    (dir_norm > cfg("near_zero_vector_norm_threshold"))
      ? Vec3(best_co_direction / dir_norm)
      : Vec3::Zero();
  ca.T2_CO_nearest = SphericalTensor::Decompose(best_co_kernel.K);
}
```

## 4 Accumulation and projection

Every accepted bond first contributes its raw asymmetric  $M_{\text{over\_r3}}$  to  $M_{\text{total}}$ . The same raw per-bond tensor may also enter one broad category total. Backbone C=O, backbone C–N, and other backbone bonds feed  $M_{\text{backbone\_total}}$ ; side-chain C=O and other side-chain bonds feed  $M_{\text{sidechain\_total}}$ ; aromatic bonds feed  $M_{\text{aromatic\_total}}$ . Disulfide, unknown, or otherwise unhandled categories still enter  $M_{\text{total}}$  after the filters, but they do not enter these broad category totals. The scalar sums are narrower: only peptide C=O, peptide C–N, side-chain C=O, and aromatic bonds add their  $f$  values.

```
M_total += kernel.M_over_r3;

case BondCategory::PeptideCO:
  co_sum += kernel.f;
  M_backbone_total += kernel.M_over_r3;
  update_nearest_CO_if_closer();
  break;

case BondCategory::PeptideCN:
  cn_sum += kernel.f;
  M_backbone_total += kernel.M_over_r3;
  update_nearest_CN_if_closer();
  break;

case BondCategory::Aromatic:
  aromatic_sum += kernel.f;
  M_aromatic_total += kernel.M_over_r3;
  break;
```

From this shared accepted-bond input the code takes two different projection paths. The category tensors first discard the asymmetric part by forming  $\frac{1}{2}(M + M^T)$ , then remove the trace, then decompose. The symmetrization discards the  $T_1$  produced by term 1. The trace subtraction sets  $T_0$  to zero before `Decompose` sees the matrix. The packed category output writes only the five  $T_2$  numbers.

```
Mat3 S = 0.5 * (M_backbone_total + M_backbone_total.transpose());
S -= (S.trace() / 3.0) * Mat3::Identity();
ca.T2_backbone_total = SphericalTensor::Decompose(S);

S = 0.5 * (M_sidechain_total + M_sidechain_total.transpose());
S -= (S.trace() / 3.0) * Mat3::Identity();
ca.T2_sidechain_total = SphericalTensor::Decompose(S);

S = 0.5 * (M_aromatic_total + M_aromatic_total.transpose());
S -= (S.trace() / 3.0) * Mat3::Identity();
```

```
ca.T2_aromatic_total = SphericalTensor::Decompose(S);
```

`mc_shielding_contribution` takes the other path on the same atom's accepted raw tensors. It decomposes  $M_{\text{total}}$  directly, without symmetrizing and without subtracting the trace first. Its  $T_0$  is the trace average of the full accepted-bond sum, its  $T_1$  is the antisymmetric part left by the first dyadic term, and its  $T_2$  is the symmetric traceless part of that same raw sum. The full nine-column output therefore reports the whole asymmetric McConnell sum, while the category tensor output reports symmetric-traceless projections. The nearest C=O and C–N blocks in the category array are different again: they decompose the stored  $K$  of the nearest accepted bond of each type, and then write only its  $T_2$  entries.

```
ca.mc_shielding_contribution =
    SphericalTensor::Decompose(M_total);    // raw M, T0 + T1 + T2

const SphericalTensor* cats[5] = {
    &ca.T2_backbone_total, &ca.T2_sidechain_total,
    &ca.T2_aromatic_total, &ca.T2_CO_nearest, &ca.T2_CN_nearest
};
```

The emitted arrays have fixed second dimensions. If no accepted nearest C=O or C–N bond is found, the corresponding distance remains `NO_DATA_SENTINEL`, 99.0 Å.

```
mc_shielding.npy    // N x 9: [T0, T1[0..2], T2[0..4]]
mc_category_T2.npy // N x 25: backbone, sidechain, aromatic, CO, CN
mc_scalars.npy     // N x 6: four scalar sums, nearest CO/CN distances
```

## 5 Shared tensor split

`SphericalTensor::Decompose` is closed-form arithmetic on a  $3 \times 3$  matrix  $\mathbf{X}$ . It does not diagonalize  $\mathbf{X}$ . The trace gives

$$T_0 = \frac{1}{3}(X_{xx} + X_{yy} + X_{zz}).$$

The antisymmetric part gives the three-vector

$$\mathbf{T}_1 = \frac{1}{2} \begin{pmatrix} X_{yz} - X_{zy} \\ X_{zx} - X_{xz} \\ X_{xy} - X_{yx} \end{pmatrix}.$$

The symmetric traceless matrix  $\mathbf{S}$  is built as

$$S_{ab} = \frac{1}{2}(X_{ab} + X_{ba}) - T_0\delta_{ab}.$$

The stored five-vector is

$$\mathbf{T}_2 = \begin{pmatrix} \sqrt{2} S_{xy} \\ \sqrt{2} S_{yz} \\ \sqrt{3/2} S_{zz} \\ \sqrt{2} S_{xz} \\ (S_{xx} - S_{yy})/\sqrt{2} \end{pmatrix}.$$

```
st.T0 = sigma.trace() / 3.0;

st.T1[0] = 0.5 * (sigma(1,2) - sigma(2,1));
st.T1[1] = 0.5 * (sigma(2,0) - sigma(0,2));
```

```

st.T1[2] = 0.5 * (sigma(0,1) - sigma(1,0));

double Sxx = sigma(0,0) - st.T0;
double Syy = sigma(1,1) - st.T0;
double Szz = sigma(2,2) - st.T0;
double Sxy = 0.5 * (sigma(0,1) + sigma(1,0));
double Sxz = 0.5 * (sigma(0,2) + sigma(2,0));
double Syz = 0.5 * (sigma(1,2) + sigma(2,1));

st.T2[0] = std::sqrt(2.0) * Sxy;
st.T2[1] = std::sqrt(2.0) * Syz;
st.T2[2] = std::sqrt(3.0 / 2.0) * Szz;
st.T2[3] = std::sqrt(2.0) * Sxz;
st.T2[4] = (Sxx - Syy) / std::sqrt(2.0);

```

The factors on the five  $T_2$  entries make the packing isometric. The  $\sqrt{2}$  on an off-diagonal entry accounts for the Frobenius double count:  $S_{xy}$  appears twice in the matrix, once as  $S_{xy}$  and once as  $S_{yx}$ , so it contributes  $2S_{xy}^2$  to  $\sum_{ab} S_{ab}^2$ . The packed entry  $\sqrt{2} S_{xy}$  squares to that same number. The diagonal factors account similarly for the two free diagonal degrees of freedom after  $S_{xx} + S_{yy} + S_{zz} = 0$ . A dot product of two stored  $T_2$  five-vectors is therefore the Frobenius inner product of their symmetric traceless matrices.

McConnell uses all three channels in `mc_shielding_contribution`. The dipolar kernel  $K$ , the nearest C=O and C–N tensors, and the broad category tensor outputs populate the  $T_2$  channel by construction; their  $T_0$  and  $T_1$  entries are not written to `mc_category_T2.npy`.

## 6 Glossary

**Dyadic product.** A grid whose rows are labeled by one vector and whose columns are labeled by another, with each cell holding the product of its two labels. In the nondegenerate McConnell kernel,  $\hat{d} \otimes \hat{b}$  has entry  $\hat{d}_a \hat{b}_b$ , and it is asymmetric unless the two directions are parallel or antiparallel. Formally, for column vectors  $u, v$ ,  $(u \otimes v)_{ab} = u_a v_b$ .

**Dipolar kernel.** The field shape of a small bar magnet — strongest and signed along its axis, weaker and reversed to the side, fading as the cube of distance (an axially symmetric, trace-zero tensor). Along its axis it has eigenvalue  $+2/r^3$ , while either broadside direction has eigenvalue  $-1/r^3$ ; the trace is zero because those three directions balance. Formally, for an axis  $\hat{u}$ , it is the symmetric traceless tensor  $(3\hat{u}_a \hat{u}_b - \delta_{ab})/r^3$ .

**Symmetric-traceless projection.** Drop a matrix's rotational swirl and its uniform scaling, and what is left is pure shape (remove the antisymmetric part and the trace average; the result is symmetric and has zero trace). The broad McConnell category totals use it by forming  $0.5(M + M^T)$ , then subtracting one third of that trace from each diagonal entry before decomposition. Formally,

$$\text{STF}(M) = \frac{1}{2}(M + M^T) - \frac{1}{3} \text{tr} \left( \frac{1}{2}(M + M^T) \right) \mathbf{I}.$$

**Near-field exclusion.** A bubble drawn around the source, sized to the real object the point kernel stands in for (an exclusion region scaled to that object). For a bond of length  $\ell$ , the per-atom McConnell path omits the candidate unless the midpoint distance is greater than  $0.5\ell$ ; equality is also omitted on that path. Formally, it is the filter  $r > \rho_{\text{nf}} \ell$  with  $\rho_{\text{nf}} = \text{near\_field\_exclusion\_ratio} = 0.5$ .

**Spherical tensor split.** A  $3 \times 3$  tensor holds three kinds of content that rotate without mixing — an orientation-free size (the scalar trace,  $T_0$ ), an axial twist from its antisymmetric part (the three  $T_1$  components), and a five-number traceless shape for the rest of the direction-dependence (the five  $T_2$  components). Here the full McConnell sum can fill all nine stored numbers, while  $K$  and the category projections contribute only the five  $T_2$  numbers that are written in the category array. Formally, it decomposes the nine Cartesian components into dimensions  $1 + 3 + 5$ , with the  $T_2$  basis normalized to preserve the Frobenius norm.